

Ed's Skills

Two skills form the core design-to-implementation workflow for Drydock and related projects.

/thinkthrough <subject>

Purpose: Didactic, subtractive design discussion. Think through a command or topic conversationally — at your altitude — without any code edits, spec changes, or plans being produced unasked.

When to use: You want to reason through a design before building it. You're not sure what you want yet. You want a collaborator who asks one question at a time and doesn't run ahead.

What it produces: A structured notes file at `docs/notes_<subject>.md` with decisions captured automatically as they're made. Sections are tagged with:

- `spec:approved` — this decision changes a behavioral contract; needs to go into the spec
- `spec:na` — design decision, no spec change needed
- `spec:applied` — already written into `Drydock_Specification.md`
- `impl:unimplemented` — code needs to be written
- `impl:implemented` — already built

Workflow: `/thinkthrough analyze` → discuss → decisions auto-captured → Close Out compacts the session → `/apply-notes analyze` to implement.

Commands:

- `/thinkthrough <subject>` — start or continue a session
 - `"close out"` or `/thinkthrough close` — compact and finish the session
 - `"halt"` or `/thinkthrough stop` — close out and exit
-

/apply-notes <subject>

Purpose: Surface all flagged items from a notes file as a numbered selectable list, then implement the ones you choose.

When to use: After a `/thinkthrough` session, when you're ready to build. You want to see all the pending code changes and spec changes, pick which to implement, and have them done in one pass.

What it surfaces:

- **CODE CHANGES** (`impl:unimplemented`) — numbered 1, 2, 3... — these are the priority; implement by editing source files, tests, prompts, Rigging templates

- **DOC CHANGES** (`spec:approved`) — lettered A, B, C... — reconcile approved decisions into `Drydock_Specification.md`; always shows diff before writing

Selection syntax: reply with numbers, letters, ranges, or keywords:

- `all code` — implement all CODE items
- `all docs` — apply all DOC items (with diff confirmation per item)
- `all` — both
- `1 3 5 A` — specific items

After implementation: flags flip (`impl:unimplemented` → `impl:implemented`, `spec:approved` → `spec:applied`), tests run, SOUNDINGS updated, commit made.

The workflow

```

/thinkthrough <subject>   ← design conversation, decisions captured
    ↓
  close out               ← compact session to notes file
    ↓
/apply-notes <subject>    ← select and implement
    ↓
  /clear                  ← start fresh for next topic

```

drydock prompt review <component>

Purpose: Evaluate one Drydock prompt contract against the authoritative specification, the matching `docs/notes_<component>.md` file, and the downstream consumer contract. This is the repeatable critique loop for prompt iteration: score it, explain why, recommend fixes, archive the result, then revise and run again.

When to use: You are iterating a prompt such as `analyze` or `plan` and want a durable, comparable review instead of an ad hoc chat assessment. Use it when the question is not “what should this command do?” but “how well will this prompt do that job?”

What it reads:

- the prompt under review from `prompts/`
- the matching working notes file in `docs/notes_<component>.md`
- the relevant slice of `docs/Drydock_Specification.md`
- the immediate consumer contract:
 - `src/drydock/analyze.py` for `analyze`

- `prompts/MANIFEST_CONTRACT.md` and `prompts/BUEPRINTS_CONTRACT.md` for plan

What it produces: A conformed review document under `docs/prompt_reviews/` plus a timestamped archive copy of the prior review. The current review includes:

- metadata header with component, command, reviewed-at timestamp, model, and rating band
- deterministic scorecard with weighted subscores and overall $x/10$
- executive assessment and general recommendation
- best plan: the recommended fix order
- findings with concrete evidence and impact
- strengths worth preserving
- open questions only when materially unresolved
- recommended prompt edits
- review method

Scoring categories:

- `spec_alignment` — does the prompt match the authoritative behavior and approved notes?
- `input_realism` — does it ask only for inputs actually injected or safely inferable?
- `output_contract_safety` — will its output satisfy the parser/consumer contract reliably?
- `analytical_effectiveness` — will it solve the real task well, not just format output?
- `ambiguity_control` — does it surface uncertainty safely without hallucinating requirements?

Rating bands:

- **Strong** — 9.0-10.0
- **Workable** — 7.5-8.9
- **Brittle** — 6.0-7.4
- **Needs Rewrite** — below 6.0

Workflow: run `drydock prompt review analyze` or `drydock prompt review plan`, read the review in `docs/prompt_reviews/`, revise the prompt or notes, and re-run. Each pass keeps the current review easy to find while archiving the previous version with timestamp and model provenance.